

The Boosting Approach to Machine Learning An Overview

Robert E. Schapire
AT&T Labs – Research
Shannon Laboratory
180 Park Avenue, Room A203
Florham Park, NJ 07932 USA
www.research.att.com/~schapire

December 19, 2001

Abstract

Boosting is a general method for improving the accuracy of any given learning algorithm. Focusing primarily on the AdaBoost algorithm, this chapter overviews some of the recent work on boosting including analyses of AdaBoost's training error and generalization error; boosting's connection to game theory and linear programming; the relationship between boosting and logistic regression; extensions of AdaBoost for multiclass classification problems; methods of incorporating human knowledge into boosting; and experimental and applied work using boosting.

1 Introduction

Machine learning studies automatic techniques for learning to make accurate predictions based on past observations. For example, suppose that we would like to build an email filter that can distinguish spam (junk) email from non-spam. The machine-learning approach to this problem would be the following: Start by gathering as many examples as possible of both spam and non-spam emails. Next, feed these examples, together with labels indicating if they are spam or not, to your favorite machine-learning algorithm which will automatically produce a classification or prediction rule. Given a new, unlabeled email, such a rule attempts to predict if it is spam or not. The goal, of course, is to generate a rule that makes the most accurate predictions possible on new test examples.

Building a highly accurate prediction rule is certainly a difficult task. On the other hand, it is not hard at all to come up with very rough rules of thumb that are only moderately accurate. An example of such a rule is something like the following: “If the phrase ‘buy now’ occurs in the email, then predict it is spam.” Such a rule will not even come close to covering all spam messages; for instance, it really says nothing about what to predict if ‘buy now’ does not occur in the message. On the other hand, this rule will make predictions that are significantly better than random guessing.

Boosting, the machine-learning method that is the subject of this chapter, is based on the observation that finding many rough rules of thumb can be a lot easier than finding a single, highly accurate prediction rule. To apply the boosting approach, we start with a method or algorithm for finding the rough rules of thumb. The boosting algorithm calls this “weak” or “base” learning algorithm repeatedly, each time feeding it a different subset of the training examples (or, to be more precise, a different distribution or weighting over the training examples¹). Each time it is called, the base learning algorithm generates a new weak prediction rule, and after many rounds, the boosting algorithm must combine these weak rules into a single prediction rule that, hopefully, will be much more accurate than any one of the weak rules.

To make this approach work, there are two fundamental questions that must be answered: first, how should each distribution be chosen on each round, and second, how should the weak rules be combined into a single rule? Regarding the choice of distribution, the technique that we advocate is to place the most weight on the examples most often misclassified by the preceding weak rules; this has the effect of forcing the base learner to focus its attention on the “hardest” examples. As for combining the weak rules, simply taking a (weighted) majority vote of their predictions is natural and effective.

There is also the question of what to use for the base learning algorithm, but this question we purposely leave unanswered so that we will end up with a general boosting procedure that can be combined with any base learning algorithm.

Boosting refers to a general and provably effective method of producing a very accurate prediction rule by combining rough and moderately inaccurate rules of thumb in a manner similar to that suggested above. This chapter presents an overview of some of the recent work on boosting, focusing especially on the AdaBoost algorithm which has undergone intense theoretical study and empirical testing.

¹A distribution over training examples can be used to generate a subset of the training examples simply by sampling repeatedly from the distribution.

Given: $(x_1, y_1), \dots, (x_m, y_m)$ where $x_i \in X$, $y_i \in Y = \{-1, +1\}$

Initialize $D_1(i) = 1/m$.

For $t = 1, \dots, T$:

- Train base learner using distribution D_t .
- Get base classifier $h_t : X \rightarrow \mathbb{R}$.
- Choose $\alpha_t \in \mathbb{R}$.
- Update:

$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$$

where Z_t is a normalization factor (chosen so that D_{t+1} will be a distribution).

Output the final classifier:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right).$$

Figure 1: The boosting algorithm AdaBoost.

2 AdaBoost

Working in Valiant’s PAC (probably approximately correct) learning model [75], Kearns and Valiant [41, 42] were the first to pose the question of whether a “weak” learning algorithm that performs just slightly better than random guessing can be “boosted” into an arbitrarily accurate “strong” learning algorithm. Schapire [66] came up with the first provable polynomial-time boosting algorithm in 1989. A year later, Freund [26] developed a much more efficient boosting algorithm which, although optimal in a certain sense, nevertheless suffered like Schapire’s algorithm from certain practical drawbacks. The first experiments with these early boosting algorithms were carried out by Drucker, Schapire and Simard [22] on an OCR task.

The AdaBoost algorithm, introduced in 1995 by Freund and Schapire [32], solved many of the practical difficulties of the earlier boosting algorithms, and is the focus of this paper. Pseudocode for AdaBoost is given in Fig. 1 in the slightly generalized form given by Schapire and Singer [70]. The algorithm takes as input a training set $(x_1, y_1), \dots, (x_m, y_m)$ where each x_i belongs to some *domain* or *instance space* X , and each *label* y_i is in some label set Y . For most of this paper, we assume $Y = \{-1, +1\}$; in Section 7, we discuss extensions to the multiclass case. AdaBoost calls a given *weak* or *base learning algorithm* repeatedly in a series

of rounds $t = 1, \dots, T$. One of the main ideas of the algorithm is to maintain a distribution or set of weights over the training set. The weight of this distribution on training example i on round t is denoted $D_t(i)$. Initially, all weights are set equally, but on each round, the weights of incorrectly classified examples are increased so that the base learner is forced to focus on the hard examples in the training set.

The base learner's job is to find a *base classifier* $h_t : X \rightarrow \mathbb{R}$ appropriate for the distribution D_t . (Base classifiers were also called rules of thumb or weak prediction rules in Section 1.) In the simplest case, the range of each h_t is binary, i.e., restricted to $\{-1, +1\}$; the base learner's job then is to minimize the *error*

$$\epsilon_t = \Pr_{i \sim D_t} [h_t(x_i) \neq y_i].$$

Once the base classifier h_t has been received, AdaBoost chooses a parameter $\alpha_t \in \mathbb{R}$ that intuitively measures the importance that it assigns to h_t . In the figure, we have deliberately left the choice of α_t unspecified. For binary h_t , we typically set

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right) \quad (1)$$

as in the original description of AdaBoost given by Freund and Schapire [32]. More on choosing α_t follows in Section 3. The distribution D_t is then updated using the rule shown in the figure. The *final* or *combined classifier* H is a weighted majority vote of the T base classifiers where α_t is the weight assigned to h_t .

3 Analyzing the training error

The most basic theoretical property of AdaBoost concerns its ability to reduce the training error, i.e., the fraction of mistakes on the training set. Specifically, Schapire and Singer [70], in generalizing a theorem of Freund and Schapire [32], show that the training error of the final classifier is bounded as follows:

$$\frac{1}{m} |\{i : H(x_i) \neq y_i\}| \leq \frac{1}{m} \sum_i \exp(-y_i f(x_i)) = \prod_t Z_t \quad (2)$$

where henceforth we define

$$f(x) = \sum_t \alpha_t h_t(x) \quad (3)$$

so that $H(x) = \text{sign}(f(x))$. (For simplicity of notation, we write $\sum_i$ and $\sum_t$ as shorthand for $\sum_{i=1}^m$ and $\sum_{t=1}^T$, respectively.) The inequality follows from the fact that $e^{-y_i f(x_i)} \geq 1$ if $y_i \neq H(x_i)$. The equality can be proved straightforwardly by unraveling the recursive definition of D_t .

Eq. (2) suggests that the training error can be reduced most rapidly (in a greedy way) by choosing α_t and h_t on each round to minimize

$$Z_t = \sum_i D_t(i) \exp(-\alpha_t y_i h_t(x_i)). \quad (4)$$

In the case of binary classifiers, this leads to the choice of α_t given in Eq. (1) and gives a bound on the training error of

$$\prod_t Z_t = \prod_t \left[2\sqrt{\epsilon_t(1-\epsilon_t)} \right] = \prod_t \sqrt{1-4\gamma_t^2} \leq \exp\left(-2\sum_t \gamma_t^2\right) \quad (5)$$

where we define $\gamma_t = 1/2 - \epsilon_t$. This bound was first proved by Freund and Schapire [32]. Thus, if each base classifier is slightly better than random so that $\gamma_t \geq \gamma$ for some $\gamma > 0$, then the training error drops exponentially fast in T since the bound in Eq. (5) is at most $e^{-2T\gamma^2}$. This bound, combined with the bounds on generalization error given below prove that AdaBoost is indeed a boosting algorithm in the sense that it can efficiently convert a true weak learning algorithm (that can always generate a classifier with a weak edge for any distribution) into a strong learning algorithm (that can generate a classifier with an arbitrarily low error rate, given sufficient data).

Eq. (2) points to the fact that, at heart, AdaBoost is a procedure for finding a linear combination f of base classifiers which attempts to minimize

$$\sum_i \exp(-y_i f(x_i)) = \sum_i \exp\left(-y_i \sum_t \alpha_t h_t(x_i)\right). \quad (6)$$

Essentially, on each round, AdaBoost chooses h_t (by calling the base learner) and then sets α_t to add one more term to the accumulating weighted sum of base classifiers in such a way that the sum of exponentials above will be maximally reduced. In other words, AdaBoost is doing a kind of steepest descent search to minimize Eq. (6) where the search is constrained at each step to follow coordinate directions (where we identify coordinates with the weights assigned to base classifiers). This view of boosting and its generalization are examined in considerable detail by Duffy and Helmbold [23], Mason et al. [51, 52] and Friedman [35]. See also Section 6.

Schapire and Singer [70] discuss the choice of α_t and h_t in the case that h_t is real-valued (rather than binary). In this case, $h_t(x)$ can be interpreted as a “confidence-rated prediction” in which the sign of $h_t(x)$ is the predicted label, while the magnitude $|h_t(x)|$ gives a measure of confidence. Here, Schapire and Singer advocate choosing α_t and h_t so as to minimize Z_t (Eq. (4)) on each round.

4 Generalization error

In studying and designing learning algorithms, we are of course interested in performance on examples *not* seen during training, i.e., in the generalization error, the topic of this section. Unlike Section 3 where the training examples were arbitrary, here we assume that all examples (both train and test) are generated i.i.d. from some unknown distribution on $X \times Y$. The generalization error is the probability of misclassifying a new example, while the test error is the fraction of mistakes on a newly sampled test set (thus, generalization error is expected test error). Also, for simplicity, we restrict our attention to binary base classifiers.

Freund and Schapire [32] showed how to bound the generalization error of the final classifier in terms of its training error, the size m of the sample, the VC-dimension² d of the base classifier space and the number of rounds T of boosting. Specifically, they used techniques from Baum and Haussler [5] to show that the generalization error, with high probability, is at most³

$$\hat{\Pr}[H(x) \neq y] + \tilde{O}\left(\sqrt{\frac{Td}{m}}\right)$$

where $\hat{\Pr}[\cdot]$ denotes empirical probability on the training sample. This bound suggests that boosting will overfit if run for too many rounds, i.e., as T becomes large. In fact, this sometimes does happen. However, in early experiments, several authors [8, 21, 59] observed empirically that boosting often does *not* overfit, even when run for thousands of rounds. Moreover, it was observed that AdaBoost would sometimes continue to drive down the generalization error long after the training error had reached zero, clearly contradicting the spirit of the bound above. For instance, the left side of Fig. 2 shows the training and test curves of running boosting on top of Quinlan’s C4.5 decision-tree learning algorithm [60] on the “letter” dataset.

In response to these empirical findings, Schapire et al. [69], following the work of Bartlett [3], gave an alternative analysis in terms of the *margins* of the training examples. The margin of example (x, y) is defined to be

$$\text{margin}_f(x, y) = \frac{yf(x)}{\sum_t |\alpha_t|} = \frac{y \sum_t \alpha_t h_t(x)}{\sum_t |\alpha_t|}.$$

²The Vapnik-Chervonenkis (VC) dimension is a standard measure of the “complexity” of a space of binary functions. See, for instance, refs. [6, 76] for its definition and relation to learning theory.

³The “soft-Oh” notation $\tilde{O}(\cdot)$, here used rather informally, is meant to hide all logarithmic and constant factors (in the same way that standard “big-Oh” notation hides only constant factors).

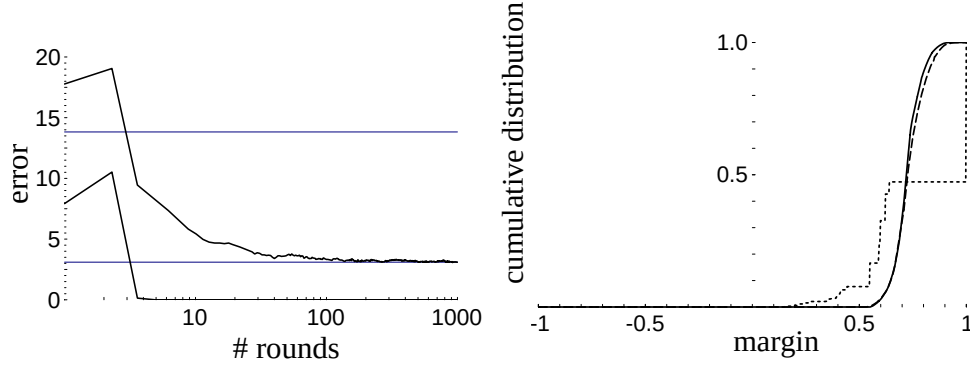


Figure 2: Error curves and the margin distribution graph for boosting C4.5 on the letter dataset as reported by Schapire et al. [69]. *Left*: the training and test error curves (lower and upper curves, respectively) of the combined classifier as a function of the number of rounds of boosting. The horizontal lines indicate the test error rate of the base classifier as well as the test error of the final combined classifier. *Right*: The cumulative distribution of margins of the training examples after 5, 100 and 1000 iterations, indicated by short-dashed, long-dashed (mostly hidden) and solid curves, respectively.

It is a number in $[-1, +1]$ and is positive if and only if H correctly classifies the example. Moreover, as before, the magnitude of the margin can be interpreted as a measure of confidence in the prediction. Schapire et al. proved that larger margins on the training set translate into a superior upper bound on the generalization error. Specifically, the generalization error is at most

$$\hat{\Pr} \left[\text{margin}_f(x, y) \leq \theta \right] + \tilde{O} \left(\sqrt{\frac{d}{m\theta^2}} \right)$$

for any $\theta > 0$ with high probability. Note that this bound is entirely independent of T , the number of rounds of boosting. In addition, Schapire et al. proved that boosting is particularly aggressive at reducing the margin (in a quantifiable sense) since it concentrates on the examples with the smallest margins (whether positive or negative). Boosting’s effect on the margins can be seen empirically, for instance, on the right side of Fig. 2 which shows the cumulative distribution of margins of the training examples on the “letter” dataset. In this case, even after the training error reaches zero, boosting continues to increase the margins of the training examples effecting a corresponding drop in the test error.

Although the margins theory gives a qualitative explanation of the effectiveness of boosting, quantitatively, the bounds are rather weak. Breiman [9], for instance,

shows empirically that one classifier can have a margin distribution that is uniformly better than that of another classifier, and yet be inferior in test accuracy. On the other hand, Koltchinskii, Panchenko and Lozano [44, 45, 46, 58] have recently proved new margin-theoretic bounds that are tight enough to give useful quantitative predictions.

Attempts (not always successful) to use the insights gleaned from the theory of margins have been made by several authors [9, 37, 50]. In addition, the margin theory points to a strong connection between boosting and the support-vector machines of Vapnik and others [7, 14, 77] which explicitly attempt to maximize the minimum margin.

5 A connection to game theory and linear programming

The behavior of AdaBoost can also be understood in a game-theoretic setting as explored by Freund and Schapire [31, 33] (see also Grove and Schuurmans [37] and Breiman [9]). In classical game theory, it is possible to put any two-person, zero-sum game in the form of a matrix $\mathbf{M}$. To play the game, one player chooses a row i and the other player chooses a column j . The loss to the row player (which is the same as the payoff to the column player) is $\mathbf{M}_{ij}$. More generally, the two sides may play randomly, choosing distributions $\mathbf{P}$ and $\mathbf{Q}$ over rows or columns, respectively. The expected loss then is $\mathbf{P}^T \mathbf{M} \mathbf{Q}$.

Boosting can be viewed as repeated play of a particular game matrix. Assume that the base classifiers are binary, and let $\mathcal{H} = \{h_1, \dots, h_n\}$ be the entire base classifier space (which we assume for now to be finite). For a fixed training set $(x_1, y_1), \dots, (x_m, y_m)$, the game matrix $\mathbf{M}$ has m rows and n columns where

$$\mathbf{M}_{ij} = \begin{cases} 1 & \text{if } h_j(x_i) = y_i \\ 0 & \text{otherwise.} \end{cases}$$

The row player now is the boosting algorithm, and the column player is the base learner. The boosting algorithm's choice of a distribution D_t over training examples becomes a distribution $\mathbf{P}$ over rows of $\mathbf{M}$, while the base learner's choice of a base classifier h_t becomes the choice of a column j of $\mathbf{M}$.

As an example of the connection between boosting and game theory, consider von Neumann's famous minmax theorem which states that

$$\max_{\mathbf{Q}} \min_{\mathbf{P}} \mathbf{P}^T \mathbf{M} \mathbf{Q} = \min_{\mathbf{P}} \max_{\mathbf{Q}} \mathbf{P}^T \mathbf{M} \mathbf{Q}$$

for any matrix $\mathbf{M}$. When applied to the matrix just defined and reinterpreted in the boosting setting, this can be shown to have the following meaning: If, for any